✓ Validation des Formulaires

Transformez vos formulaires HTML avec FormValidateController

Une approche universelle pour tous les écosystèmes JavaScript

⚠ Le Problème Que Tout Développeur Rencontre

Avez-vous déjà passé des heures à déboguer la validation de formulaires ? Si vous développez avec JavaScript, vous avez probablement rencontré ces frustrations :

Les Défis Réels :

- ◍ **Code répétitif :** Réécrire la même logique de validation pour chaque formulaire
- ◍ **Gestion des erreurs complexe :** Coordonner les messages d'erreur sur plusieurs champs
- ◍ **Incompatibilité cross-écosystème :** React, Vue, Angular... chacun sa façon
- ◍ **Validation asynchrone :** Vérifier les emails, les domaines... sans bloquer l'interface
- ◍ **Fichiers uploads :** Valider les images, vidéos, documents sans perdre les cheveux
- ◍ **Accessibilité :** Informer les utilisateurs des erreurs de manière claire et structurée

Le résultat ? Des heures perdues, des bugs en production, des validations incomplètes. Et tout cela pour un problème qui devrait être résolu une seule fois.

La Solution : FormValidateController

Rencontrez FormValidateController :

Une bibliothèque complète, **agnostique du framework**, qui gère la validation de formulaires HTML de manière déclarative et centralisée.

FormValidateController est plus qu'une simple bibliothèque de validation. C'est un système complet conçu pour :

Déclaratif

Configurez la validation directement dans votre HTML avec des attributs data

Ultra-Rapide

Validation instantanée sans surcharge performante

Universal

Fonctionne avec jQuery, VanillaJS, React, Vue, Angular...

15+ Types

Email, tel, URL, date, fichiers, FQDN, et bien plus

Sécurisé

Détection de spoofing, validation de signatures de fichiers

Responsive

Fonctionne sur tous les appareils et navigateurs

🎁 Installation Ultra-Simple

Via NPM

```bash
npm install @wlindabla/form_validator
```

Via Yarn

```bash
yarn add @wlindabla/form_validator
```

Via PNPM

```bash
pnpm add @wlindabla/form_validator
```

✔ **C'est tout !** Installez, importez, et commencez à valider.

🔧 Utilisation avec jQuery (La Manière Classique)

Si vous utilisez encore jQuery, bonne nouvelle : ça fonctionne parfaitement !

Étape 1 : Structure HTML

```html
<form class="form-validate"> <div class="form-group"> <label>Email</label>
<input type="email" id="email" name="email" data-event-validate-blur="blur"
placeholder="votre@email.com" required /> </div> <div class="form-group">
<label>Mot de passe</label> <input type="password" id="password" name="password"
data-event-validate-input="input" data-min-length="8" placeholder="Minimum 8
caractères" required /> </div> <button type="submit">S'inscrire</button> </form>
```

Étape 2 : Initialisation jQuery

```javascript
import { FormValidateController } from '@wlindabla/form_validator';
jQuery(function() { // Initialiser le contrôleur const formValidate = new
FormValidateController('.form-validate'); const $form = formValidate.form; const
idsBlur = formValidate.idChildrenUsingEventBlur; // Validation au blur (email)
idsBlur.forEach(id => { $(`#${id}`).on('blur', async () => { await
formValidate.validateChildrenForm( document.getElementById(id) ); }); }); //
Gestion des erreurs $form.on('field:validation:failed', (e) => { const { name,
message } = e.detail; $(`#${name}`) .addClass('is-invalid') .after(`<span
class="error">${message}</span>`); }); // Gestion du succès
$form.on('field:validation:success', (e) => { const { name } = e.detail;
$(`#${name}`) .removeClass('is-invalid') .siblings('.error').remove(); }); //
Soumission du formulaire $form.on('submit', async (e) => { e.preventDefault();
await formValidate.autoValidateAllFields(); console.log('✅ Validation complète
!'); }); });
```

> ✅ **Résultat :** Votre formulaire valide en temps réel avec gestion automatique des erreurs !

⚙️ Utilisation avec VanillaJS (Moderne & Pur)

Vous préférez rester 100% pur JavaScript ? C'est possible et même recommandé !

Structure HTML

```html
<form class="form-validate" id="registerForm"> <input type="text"
id="username" name="username" data-event-validate-blur="blur" data-min-length="3"
placeholder="Nom d'utilisateur" required /> <input type="tel" id="phone"
name="phone" data-event-validate-change="change" data-default-country="BJ"
placeholder="+229 XX XX XX XX" /> <input type="file" id="avatar" name="avatar"
media-type="image" data-event-validate-change="change" data-maxsize-file="2" />
<button type="submit">Valider</button> </form>
```

Code VanillaJS

```javascript
import { FormValidateController } from '@wlindabla/form_validator';
// Initialiser const formValidate = new FormValidateController('.form-validate');
const form = document.getElementById('registerForm'); // Récupérer les IDs par
événement const blurFields = formValidate.idChildrenUsingEventBlur; const
changeFields = formValidate.idChildrenUsingEventChange; // Setup événements blur
blurFields.forEach(fieldId => { const field = document.getElementById(fieldId);
field?.addEventListener('blur', async () => { await
formValidate.validateChildrenForm(field); }); }); // Setup événements change
changeFields.forEach(fieldId => { const field = document.getElementById(fieldId);
field?.addEventListener('change', async () => { await
formValidate.validateChildrenForm(field); }); }); // Gestion des erreurs
form.addEventListener('field:validation:failed', (e) => { const { name, message }
= e.detail; const field = document.getElementById(name); const errorEl =
document.createElement('span'); errorEl.className = 'error-message';
errorEl.textContent = message; field.classList.add('is-invalid');
field.after(errorEl); }); // Gestion du succès
form.addEventListener('field:validation:success', (e) => { const { name } =
e.detail; const field = document.getElementById(name);
field.classList.remove('is-invalid'); field.nextElementSibling?.remove(); }); //
```

⚛ Utilisation avec React (Moderne & Réactif)

React lovers, voici comment intégrer FormValidateController avec les hooks React :

Composant React avec Hooks

```jsx
import React, { useRef, useEffect, useState } from 'react'; import {
FormValidateController } from '@wlindabla/form_validator'; export function
RegistrationForm() { const formRef = useRef(null); const [validator,
setValidator] = useState(null); const [errors, setErrors] = useState({});
useEffect(() => { if (!formRef.current) return; // Initialiser le contrôleur
const controller = new FormValidateController(formRef.current);
setValidator(controller); // Écouter les erreurs const handleError = (e) => {
setErrors(prev => ({ ...prev, [e.detail.name]: e.detail.message })); }; //
Écouter le succès const handleSuccess = (e) => { setErrors(prev => { const
updated = { ...prev }; delete updated[e.detail.name]; return updated; }); };
formRef.current.addEventListener('field:validation:failed', handleError);
formRef.current.addEventListener('field:validation:success', handleSuccess);
return () => { formRef.current?.removeEventListener('field:validation:failed',
handleError); formRef.current?.removeEventListener('field:validation:success',
handleSuccess); }; }, []); const handleSubmit = async (e) => {
e.preventDefault(); if (validator) { await validator.autoValidateAllFields(); if
(Object.keys(errors).length === 0) { console.log('✅ Formulaire valide !'); } }
}; return ( <form ref={formRef} className="form-validate" onSubmit={handleSubmit}
> <div className="form-group"> <label>Email</label> <input type="email"
```

✅ **Avantages React :** État géré avec hooks, re-render automatique, intégration seamless avec votre écosystème React.

🌍 Support Multi-Écosystème JavaScript

Ce qui rend FormValidateController **vraiment unique**, c'est sa compatibilité avec tous les écosystèmes JavaScript modernes :

⚛ React	💚 Vue.js	🅰 Angular	🔗 jQuery
⚙ VanillaJS	🚀 Next.js	🔥 Svelte	📚 Node.js

Pourquoi C'est Possible ?

Framework-agnostique

Utilise des standards web : HTML5, attributs data, événements personnalisés

Zero dépendances

Aucune dépendance externe. jQuery est obligatoire pour la compatibilité des anciens navigateurs pour les Adapter de Dom.

Exportations modulaires

Importez uniquement ce dont vous avez besoin

Événements standard

Communique via événements DOM natifs

Cas d'Usage : Vue.js

Même approche que React : initialiser dans un hook (en l'occurrence `mounted`), écouter les événements :

```vue
<template> <form ref="formElement" class="form-validate"> <input type="email" name="email" data-event-validate-blur="blur" /> <button type="submit">Soumettre</button> </form> </template> <script> import { FormValidateController } from '@wlindabla/form_validator'; export default { mounted() { const controller = new FormValidateController(this.$refs.formElement); this.$refs.formElement.addEventListener( 'field:validation:failed', (e) => console.log('Erreur:', e.detail.message) ); } } </script>
```

🚀 Fonctionnalités Avancées Incluses

15+

Types de Champs

100%

Async Ready

0	∞
Dépendances	Cas d'Usage

Tous les Types de Champs Supportés :

Type	Exemple	Validations Incluses
Text	John Doe	Min/Max length, Regex, HTML escape
Email	user@example.com	RFC 2822, Affichage nom, FQDN, Whitelist/Blacklist
URL	https://api.example.com	Protocole, Localhost, IP, Query params, Fragments
Téléphone	+229 01 67 25 18 86	Format international, Code pays, Formatage auto
Date	2024/12/25	Format personnalisé, Min/Max, Passé/Futur
Image	photo.jpg	Signature fichier, Dimensions, MIME, Détection spoofing
Vidéo	video.mp4	Format, Dimensions, Taille, Métadonnées
Document	report.pdf	PDF, Word, Excel, CSV, Signature fichier
Mot de passe	SecureP@ss123	Complexité, Scoring, Caractères requis
Nombre	42	Min, Max, Step, Format
FQDN	example.com	TLD, Hyphens, Underscores, Wildcard
Select	option_a	Validation options, Whitelist/Blacklist
Checkbox	Multi-select	Min/Max sélections, Requis

Radio	Single select	Sélection obligatoire
Textarea	Message long	Min/Max length, Regex, HTML escape

Fonctionnalités de Sécurité :

- **Détection de spoofing :** Valide les signatures hexadécimales des fichiers
- **Validation MIME type :** Vérifie que le type réel correspond au type déclaré
- **Échappement HTML/PHP :** Prévient les injections XSS
- **Validation email :** RFC 2822 complète avec détection de domaines
- **Validation URL :** Détecte les faux protocoles et domaines malveillants

---

📋 Exemple Complet : Formulaire d'Inscription Réaliste

Voici un formulaire complet avec tous les types de champs courants :

HTML : Structure Déclarative

```html
<form class="form-validate" id="registerForm"> <!-- Informations Personnelles --> <fieldset> <legend>Informations Personnelles</legend> <input type="text" id="username" name="username" data-min-length="3" data-max-length="20" data-event-validate-blur="blur" placeholder="Nom d'utilisateur" required /> <input type="email" id="email" name="email" data-event-validate-blur="blur" placeholder="votre@email.com" required /> <input type="tel" id="phone" name="phone" data-default-country="BJ" data-event-validate-blur="blur" placeholder="+229 XX XX XX XX" required /> </fieldset> <!-- Mot de passe --> <fieldset> <legend>Sécurité du Compte</legend> <input type="password" id="password" name="password" data-event-validate-input="input" data-min-length="12" data-upper-case-allow="true" data-number-allow="true" data-symbol-allow="true" data-enable-scoring="true" placeholder="Minimum 12 caractères" required /> <input type="password" id="confirmPassword" name="confirmPassword" data-event-validate-input="input" data-min-length="12" placeholder="Confirmez le mot de passe" required /> </fieldset> <!-- Préférences --> <fieldset> <legend>Préférences</legend> <!-- Select --> <select id="country" name="country" data-event-validate-change="change" required > <option value="">-- Sélectionner
```

```
un pays --</option> <option value="BJ">Bénin</option> <option
```

JavaScript : Orchestration Simple

```javascript
import { FormValidateController } from '@wlindabla/form_validator';

// Initialiser const formValidate = new FormValidateController('#registerForm');
const form = document.getElementById('registerForm'); // Configuration globale
const successClass = 'is-valid'; const errorClass = 'is-invalid'; // Gestion des
erreurs form.addEventListener('field:validation:failed', (e) => { const { name,
message } = e.detail; const field = document.getElementById(name);
field?.classList.add(errorClass); field?.classList.remove(successClass); //
Afficher le message d'erreur const errorEl = field?.nextElementSibling; if
(errorEl?.classList.contains('error-text')) { errorEl.textContent = message; }
else { const newError = document.createElement('small'); newError.className =
'error-text'; newError.textContent = message; field?.after(newError); } }); //
Gestion du succès form.addEventListener('field:validation:success', (e) => {
const { name } = e.detail; const field = document.getElementById(name);
field?.classList.remove(errorClass); field?.classList.add(successClass);
field?.nextElementSibling?.remove(); }); // Soumission
form.addEventListener('submit', async (e) => { e.preventDefault(); // Valider
tous les champs await formValidate.autoValidateAllFields(); // Vérifier si tout
est valide const hasErrors = document.querySelectorAll( `.${errorClass}` ).length
> 0; if (!hasErrors) { console.log('✅ Formulaire valide !'); // Soumettre via
```

CSS : Styling Optionnel

```css
/* Styling des champs */ input, select, textarea { padding: 12px; border:
2px solid #ddd; border-radius: 6px; font-size: 1em; transition: all 0.3s; } /*
Champ valide */ input.is-valid, select.is-valid, textarea.is-valid { border-color:
#28a745; background: #f0f9f7; } /* Champ invalide */ input.is-invalid, select.is-
invalid, textarea.is-invalid { border-color: #dc3545; background: #ffe5e5; } /*
Messages d'erreur */ .error-text { display: block; color: #dc3545; font-size:
0.85em; margin-top: 5px; font-weight: 600; } /* Focus états */ input:focus,
select:focus, textarea:focus { outline: none; box-shadow: 0 0 0 3px rgba(102, 126,
234, 0.2); }
```

📁 Cas d'Usage Réels

E-Commerce

Validation de formulaires de paiement, adresses de livraison, uploads de photos produit

Santé

Validation de données sensibles, dates de naissance, numéros de dossier

Social Media

Inscription utilisateur, uploads de profil, validation d'emails

SaaS

Formulaires de configuration, imports de données, uploads de documents

Éducation

Inscription d'étudiants, uploads de certificats, validation de diplômes

Finance

Formulaires KYC, validation d'adresses, uploads de pièces d'identité

Performance & Sécurité

Optimisations de Performance :

- **Validation instantanée :** Pas de délai entre la saisie et la validation
- **Caching intelligent :** Cache les options de validation pour plusieurs passages
- **Événements natifs :** Utilise les événements DOM standards
- **Pas de polling :** Basé sur les événements, pas de boucles d'attente
- **Lazy loading :** Les validateurs sont chargés à la demande

Mesures de Sécurité :

- **XSS Prevention :** Échappement automatique des tags HTML/PHP
- **File Spoofing Detection :** Valide les signatures hexadécimales

- **MIME Type Verification :** Vérifie la cohérence du type MIME
- **Input Sanitization :** Nettoie automatiquement les entrées
- **RFC Compliance :** Respecte les standards (RFC 2822 pour email)

> ⚠️ **Important :** Validez **toujours** côté serveur également ! La validation front-end est pour l'UX, pas pour la sécurité. Les utilisateurs malveillants peuvent contourner le JavaScript.

👨‍💻 Avantages pour les Développeurs

⏱️ Économise du Temps

Pas besoin de réécrire la validation pour chaque projet

🧩 Réduit la Complexité

Une seule approche pour tous les frameworks

🔄 Maintenable

Mise à jour centralisée, pas de code dupliqué

🧪 Testable

API claire, facile à tester en unit tests

📚 Bien Documentée

Documentation complète avec exemples

🤝 Communauté

Support actif et contributions bienvenues

⚖️ Comparaison : FormValidateController vs Alternatives

Critère	FormValidateController	Alternatives
Framework Agnostique	✓ **Oui**	Généralement spécifique
Zero Dépendances	✓ **Oui**	Souvent dépendances
Types Supportés	✓ **15+**	Généralement 8-10
Upload Fichiers	✓ **Complet**	Support limité
Détection Spoofing	✓ **Oui**	✗ **Non**
Validation Async	✓ **Native**	Dépend de l'outil
Configuration HTML	✓ **Oui**	Généralement JS
Multi-Langue	✓ **Oui**	Limité

🎯 Conclusion : L'Avenir de la Validation

FormValidateController change la façon dont les développeurs JavaScript pensent à la validation des formulaires. Plus besoin de choisir entre frameworks, plus besoin de réécrire le même code, plus besoin de gérer les cas spéciaux.

✨ **Une seule validation pour tous vos projets :**

- ✓ jQuery, React, Vue, Angular, Svelte...
- ✓ 15+ types de champs incluant fichiers
- ✓ Validation asynchrone intégrée
- ✓ Détection de spoofing avancée
- ✓ Configuration déclarative en HTML

- ✓ Zero dépendances, 100% Web Standards

Prêt à commencer ?

L'installation prend moins d'une minute. Les exemples sont prêts à copier-coller. La documentation est complète.

📖 Voir la Documentation 🎁 Installer via NPM

👨‍💻 À Propos de l'Auteur

AGBOKOUDJO Franck est un développeur passionné par l'open source et les solutions réutilisables. Il a créé FormValidateController après des années de frustration avec les solutions fragmentées de validation.

Son mission : **simplifier la vie des développeurs** en fournissant des outils robustes, documentés et universels.

GitHub NPM Email

📚 Ressources Supplémentaires

Documentation Complète :

- Repository GitHub

- Package NPM
- Wiki & Guides

Exemples de Code :

- Form Vanilla JS simple
- React Component avancé
- Vue.js avec Composition API
- Angular avec Services
- Next.js avec Serveur Actions

Prochainement :

Dans les prochains articles, nous explorerons :

- 🔐 **Validation côté serveur :** Node.js & Express
- 🗄️ **Intégration Base de données :** Validation de contraintes
- 🌐 **Internationalisation :** Messages d'erreur en 50+ langues
- ⚙️ **Plugins personnalisés :** Créez vos propres validateurs

---

FormValidateController

La solution universelle pour valider les formulaires HTML

© 2025 AGBOKOUDJO Franck |internationaleswebservices@gmail.com| Open Source | MIT License

⭐ Star sur GitHub 📦 NPM Package 🔗 LinkedIn

Article publié le 2025 | À partager sur LinkedIn & Facebook